Memory Virtualization

- Segmentation and Paging -

Youngjae Kim (Ph.D.)

Sogang University

Linux 운영체제 및 응용 (GITF315-01)

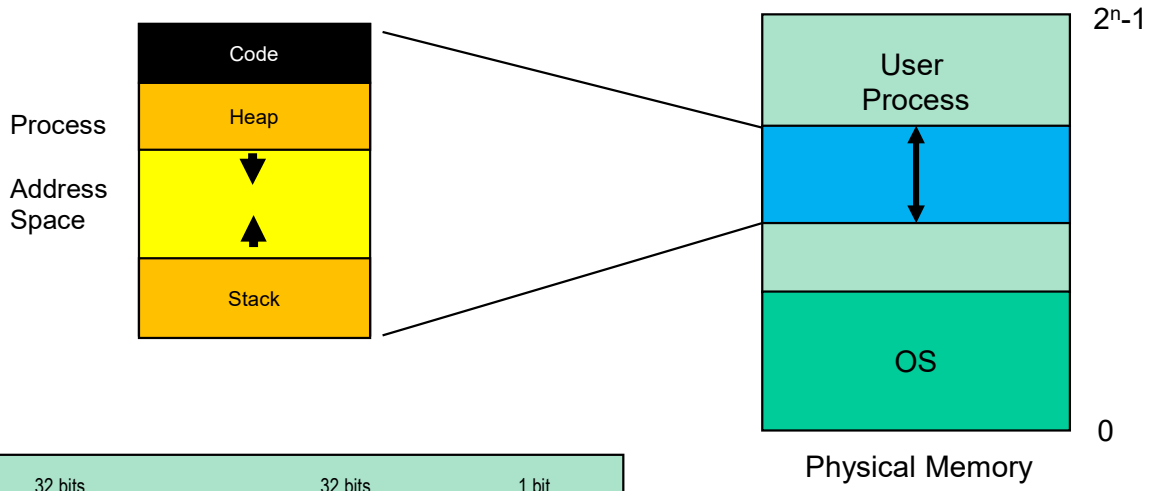
R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 2nd ed. Arpaci-Dusseau Books, 2018. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

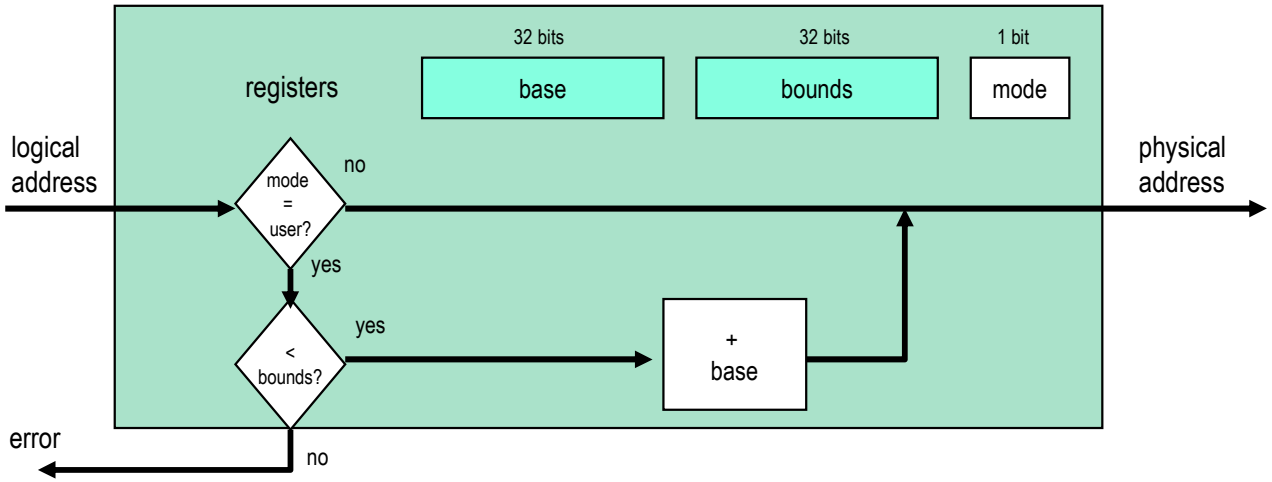
Segmentation

Base and Bounds DISADVANTAGES

- **Disadvantages**
  - Each process must be allocated contiguously in physical memory
    - Must allocate memory that may not be used by process

계속하여 메모리에 여러 프로세스를 생성/소멸하다 보면 구멍(frag)들이 생기는데, 이러한 frag들을 합치면 할당이 가능하지만 쪼개져 있어 할당이 불가능한 상황이 생김..
=> 외부 단편화





Segmentation

외부 단편화 문제를 해결하고자.
segment라는 논리적 단위로 쪼개서
메모리를 할당하게 됨.

What is Segmentation?

- The idea was born in the early 1960's.
- A segment is simply a **contiguous region** of the address space.
- Instead of having just one base-and-bounds pair in the MMU, segmentation assigns **a separate base and bounds pair for each logical segment**.
- A process typically has three or four logically distinct segments: **Code (Text), Data, Stack, and Heap**.

Why is Segmentation?

- Segmentation allows the OS to place each of these segments (Code, Stack, Heap, etc.) in different areas of physical memory, preventing physical memory from being wasted by unused portions of a large, continuous virtual address space.

Segmented Addressing

- **Benefits of segmentation**

  Each segment can independently:

  - Be placed anywhere in physical memory
  - Grow or shrink dynamically
  - Be protected separately (distinct read/write/execute permission bits)

- **Addressing with Segmentation**

  A process generates an address by specifying:

  - A **segment**, and
  - An **offset** within that segment.

- **How does a process designate a particular segment?**

  The segment can be encoded directly in the logical address:

  - The **upper bits** of the logical address identify the **segment**.
  - The **lower bits** represent the **offset** within that segment.

When Bits Are Not Enough

- **Problem**
  - What if the address space is small and there are not enough bits to encode the segment?
- **Solution: Implicit Segment Selection**

  CPU selects the segment according to the memory reference type:
  - **Instruction fetch** → Code Segment (CS)
  - **Stack operations** (push, pop, call, return) → Stack Segment (SS)
  - **Data loads/stores** → Data Segment (DS) depending on addressing mode

Type of memory reference	Segment automatically selected
Instruction fetch	Code segment (CS)
Stack operations (push/pop)	Stack segment (SS)
Data load/store	Data segment (DS)

Segmentation Implementation

- **MMU and Segment Table**
  - The MMU maintains a Segment Table for each process.
  - Each segment has its own:
    - **Base address**
    - **Bounds (limit)**
    - **Protection bits** (read/write/execute)

- **Segment Table**

Segment	Base	Bounds	R W
0	0x2000	0x6ff	1 0
1	0x0000	0x4ff	1 1
2	0x3000	0xfff	1 1
3	0x0000	0x000	0 0

remember:
1 hex digit->4 bits

Example

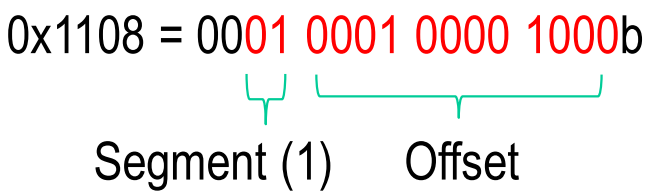
- **14-bit logical address, 4 segments**
  - How many bits are needed to identify the segment?
  - How many bits are left for the offset within the segment?

Segment	Base	Bounds	R W
0	0x2000	0x6ff	1 0
1	0x0000	0x4ff	1 1
2	0x3000	0xfff	1 1
3	0x0000	0x000	0 0

remember:
1 hex digit->4 bits

Translate logical addresses (in hex) to physical addresses

0x0240:
0x1108:
0x265c:
0x3002:



Segment Selection

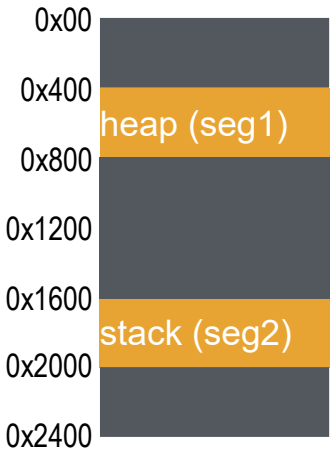
```
1   // get top 2 bits of 14-bit VA
2   Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3   // now get offset
4   Offset = VirtualAddress & OFFSET_MASK
5   if (Offset >= Bounds[Segment])
6        RaiseException(PROTECTION_FAULT)
7   else
8        PhysAddr = Base[Segment] + Offset
9        Register = AccessMemory(PhysAddr)
```

- SEG_MASK = 0x3000(**11000000000000**)
- SEG_SHIFT = 12
- OFFSET_MASK = 0xFFF (**00111111111111**)

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

remember:
- AND with 0 clears the value
- AND with 1 preserves the value

Visual Interpretation

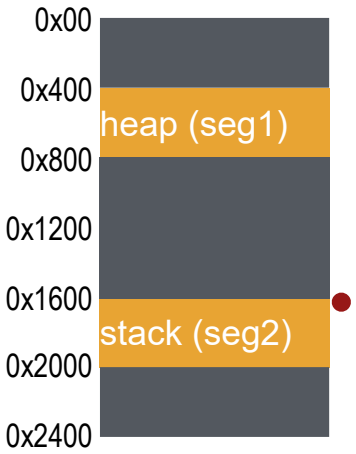


Virtual (hex)	Physical
load 0x2010, R1	

Segment numbers:

0: code+data
1: heap
2: stack

Visual Interpretation

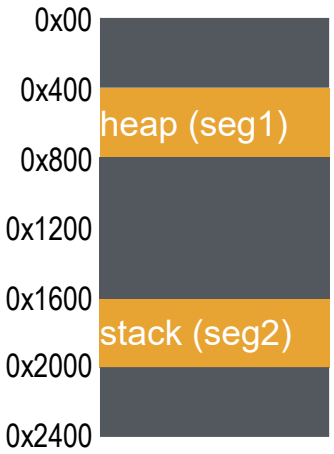


Virtual (hex)	Physical
load 0x2010, R1	0x1600 + 0x010 = 0x1610

Segment numbers:

0: code+data
1: heap
2: stack

Visual Interpretation

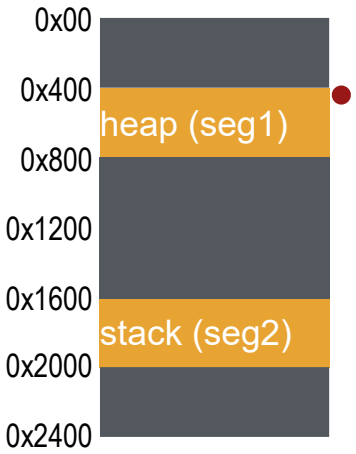


Virtual (hex)	Physical
load 0x2010, R1	0x1600 + 0x010 = 0x1610
load 0x1010, R1	

Segment numbers:

0: code+data
1: heap
2: stack

Visual Interpretation

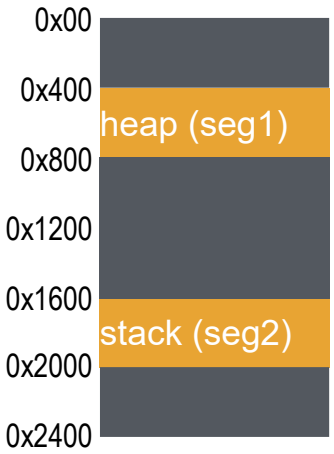


Virtual (hex)	Physical
load 0x2010, R1	0x1600 + 0x010 = 0x1610
load 0x1010, R1	0x400 + 0x010 = 0x410

Segment numbers:

0: code+data
1: heap
2: stack

Visual Interpretation



Virtual (hex)	Physical
load 0x2010, R1	0x1600 + 0x010 = 0x1610
load 0x1010, R1	0x400 + 0x010 = 0x410
load 0x1100, R1	

Segment numbers:

0: code+data
1: heap
2: stack

Visual Interpretation

Virtual (hex)	Physical
load 0x2010, R1	0x1600 + 0x010 = 0x1610
load 0x1010, R1	0x400 + 0x010 = 0x410
load 0x1100, R1	0x400 + 0x100 = 0x500

Segment numbers:

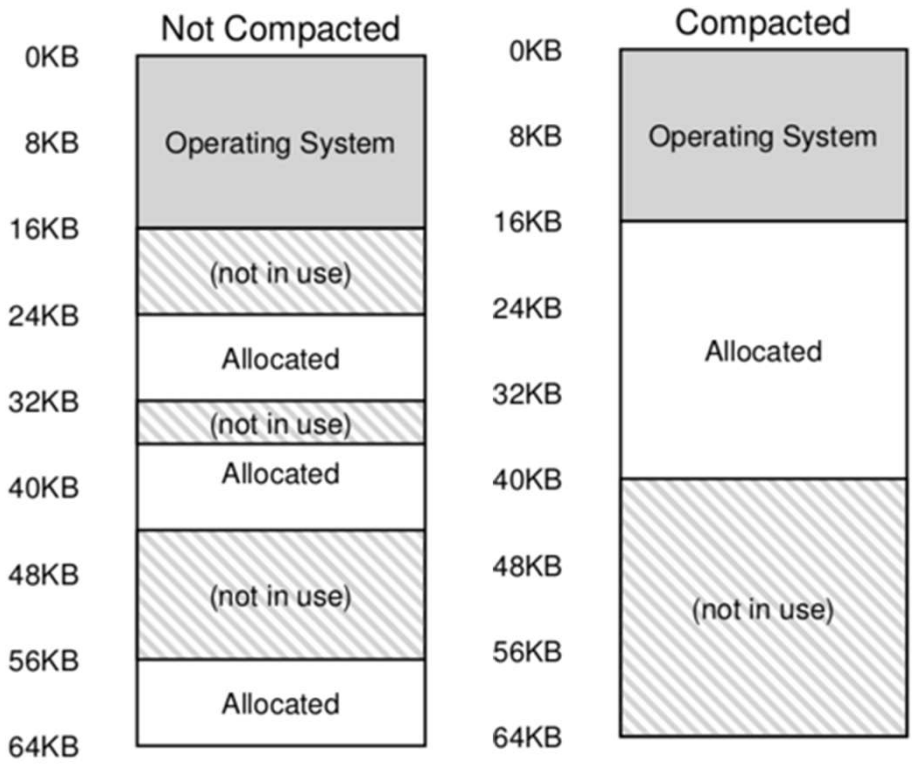
0: code+data
1: heap
2: stack

Advantages of Segmentation

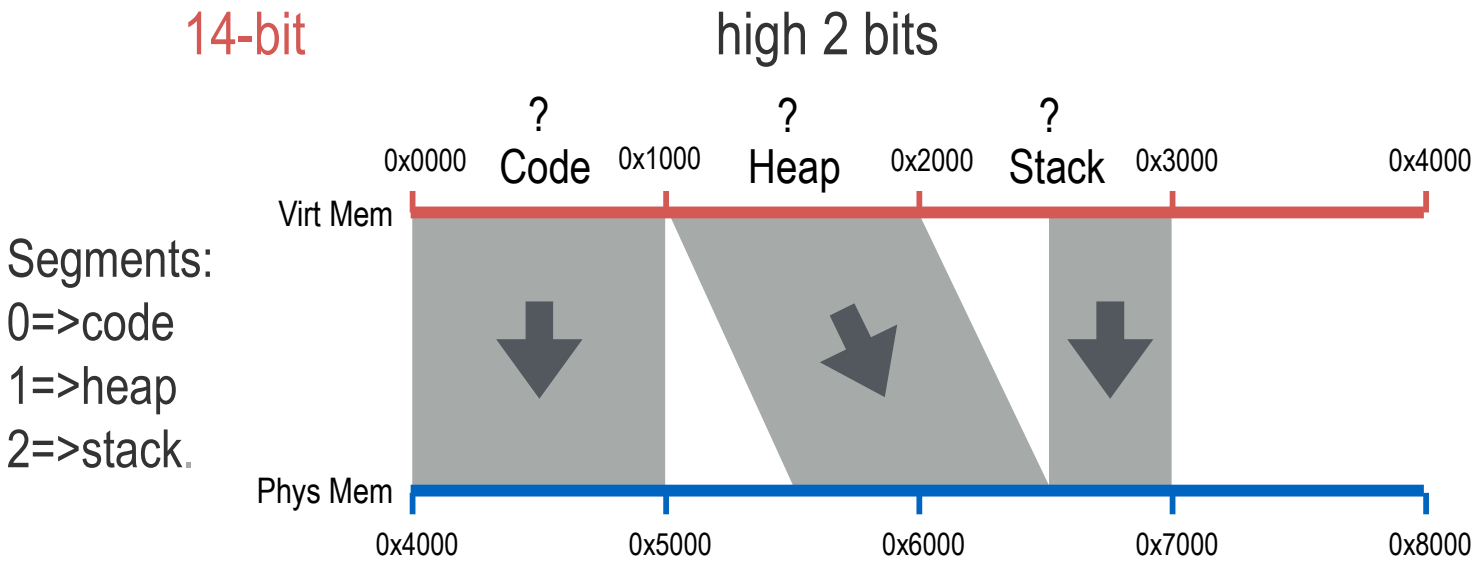
- **Enables sparse allocation of the address space**
  - Stack and heap can **grow independently**.
    - **Heap**: When no free blocks are available, the dynamic memory allocator **explicitly** requests more memory from the OS (e.g., `malloc` → `sbrk()` in UNIX).
    - **Stack**: The OS detects references beyond the current stack boundary and **implicitly** extends the stack segment.
- **Provides different protection levels for different segments**
  - For example, the code segment can be marked read-only.
  - Supports sharing of selected segments
  - Useful for shared libraries or shared memory regions.
- **Allows dynamic relocation of each segment**
  - Each segment can be placed anywhere in physical memory as long as its base and bounds are respected.

Disadvantages of Segmentation

- **Each segment must be allocated contiguously**
  - **Large segments** may not fit into available physical memory blocks, causing **external fragmentation**.
  - Compaction can reduce fragmentation but is expensive and not always feasible.
- **Will fix this issue using paging**
  - Paging removes the need for contiguous physical allocation….



Review: Segmentation



Where doe segment table live?

All registers, MMU

Seg	Base	Bounds
0	0x4000	0xfff
1	0x5800	0xfff
2	0x6800	0x7ff

Review: Memory Access

```
0x0010:  movl   0x1100, %edi
0x0013:  addl $0x3, %edi
0x0019:  movl   %edi, 0x1100
```

%rip: 0x0010

Seg	Base	Bounds
0	0x4000	0xfff
1	0x5800	0xfff
2	0x6800	0x7ff

Physical Memory Accesses?

1) Fetch instruction at logical addr 0x0010
   - Physical addr: 0x4010

   Exec, load from logical addr 0x1100
   - Physical addr: 0x5900

2) Fetch instruction at logical addr 0x0013
   - Physical addr: 0x4013

   Exec, no load

3) Fetch instruction at logical addr 0x0019
   - Physical addr: 0x4019

   Exec, store to logical addr 0x1100
   - Physical addr: 0x5900

Total of 5 memory references (3 instruction fetches, 2 movl)

Paging

Paging

페이징은 프로세스의 논리 주소를 고정 크기로 잘라 MMU를 통해 물리 주소로 변환함으로써 외부 단편화를 해결하는 방식이며, 전체 페이지 테이블은 DRAM에 저장하되 속도 저하를 막기 위해 자주 쓰는 변환 정보만 MMU 내의 TLB에 캐싱하여 처리하는 시스템이다.

What is Paging?

- **Paging splits the address space into fixed-size units called pages.**
  - Segmentation uses variable-sized logical segments (code, stack, etc.).
- Physical memory is partitioned into fixed-size blocks called **page frames**.
- **Each process maintains a page table**, which the hardware uses to translate virtual addresses into physical addresses.

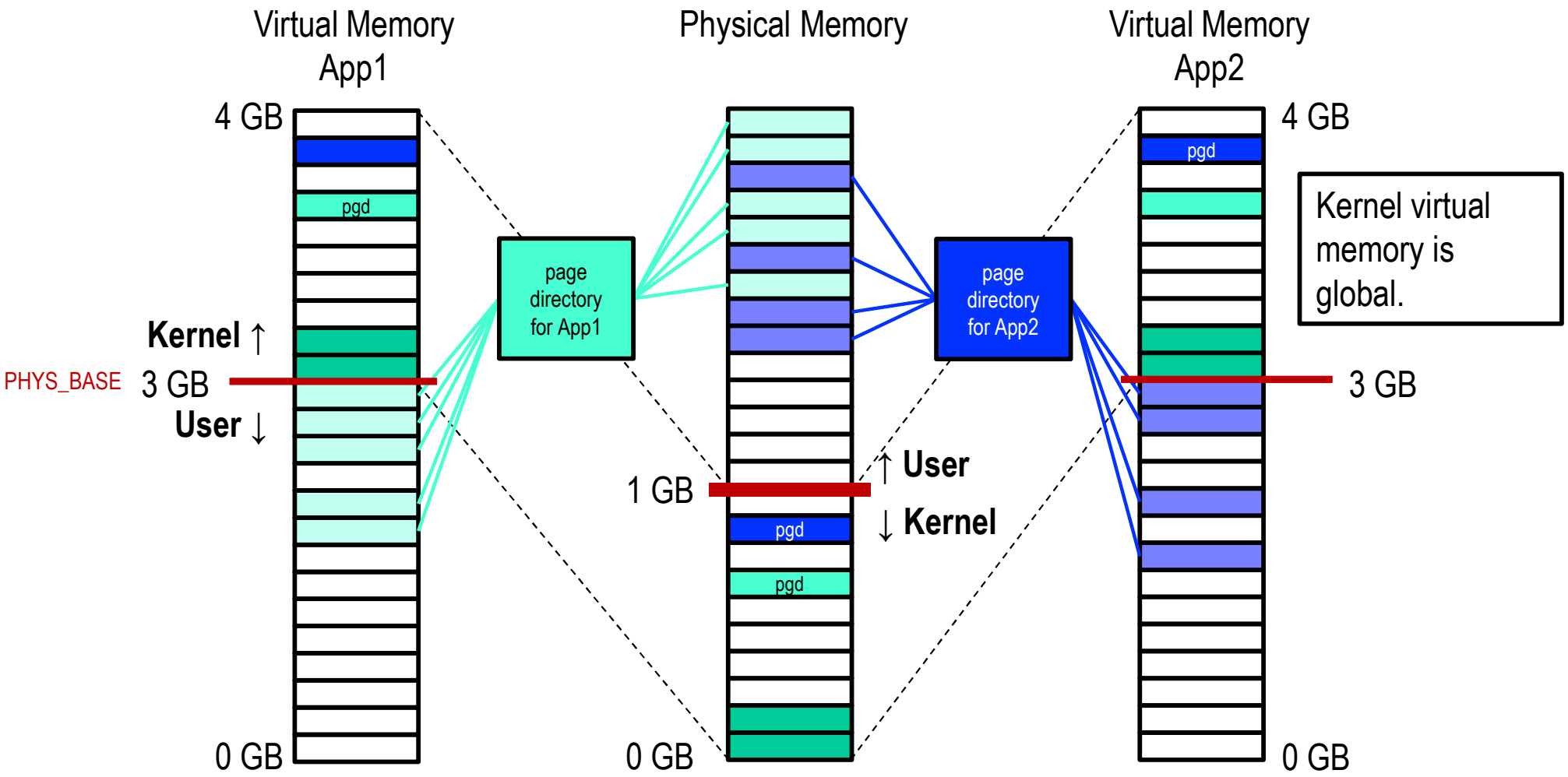
Why is Paging?

- **Eliminates external fragmentation** because physical memory does not need to be allocated contiguously.
- **Simplifies memory management** since all pages are the same size.
- **Enables flexible virtual-to-physical mapping**, improving multiprogramming and memory utilization.
- **Supports demand paging**, allowing pages to be loaded only when needed.

Virtual Memory: Paging

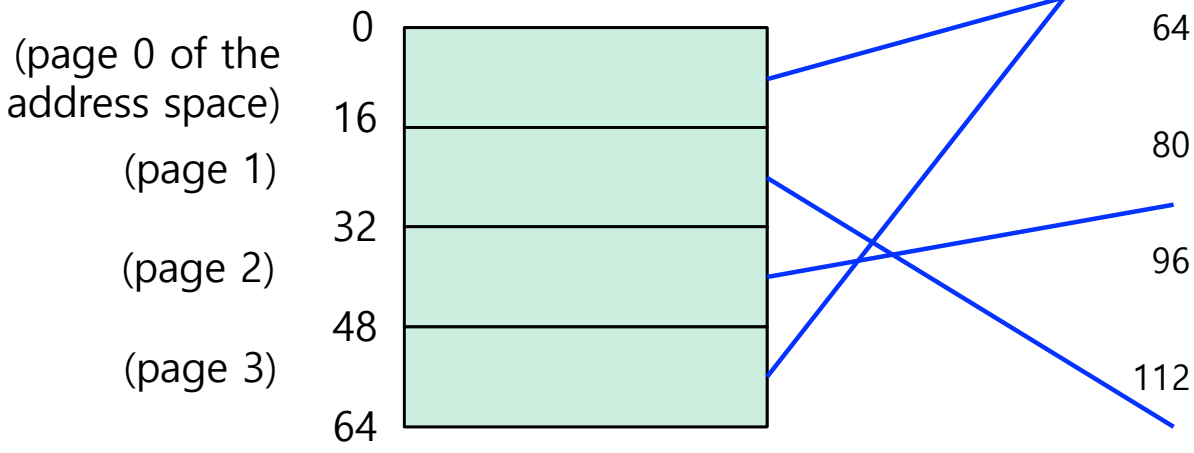
4kb 단위로 쪼갰을 때 그에 대한 인덱싱 테이블이 너무 커져서 MMU에 다 넣을 수 없음.. 그래서 paging이 필요한거임.

↳ Page Table

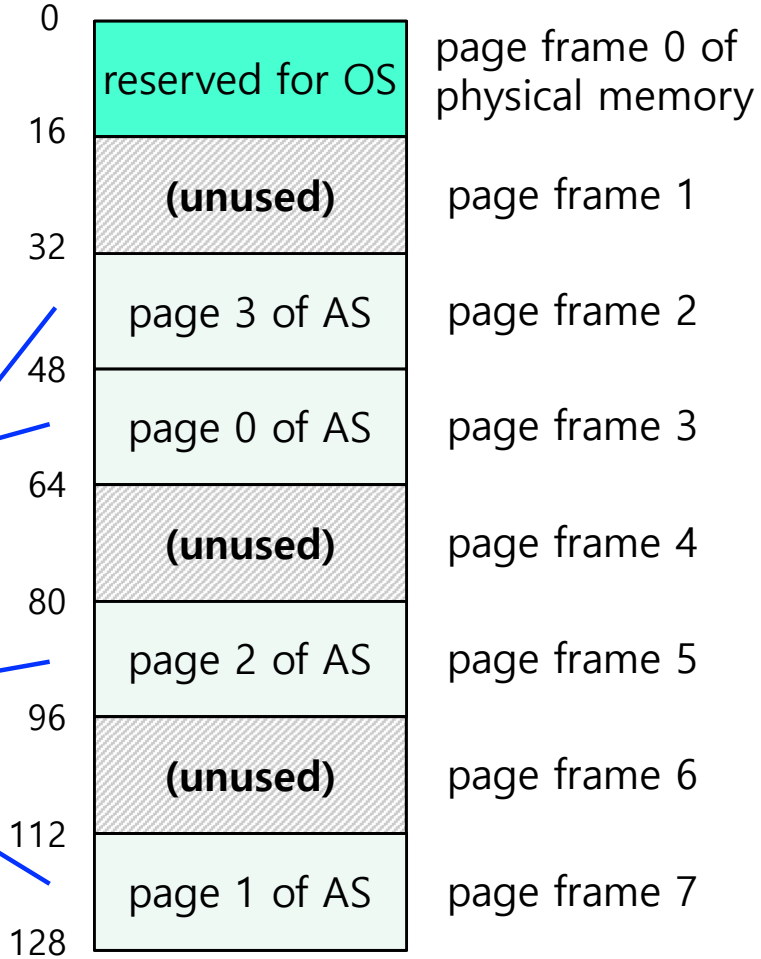


Example: A Simple Paging

- **128-byte physical memory with 16 bytes page frames**
- **64-byte address space with 16 bytes pages**



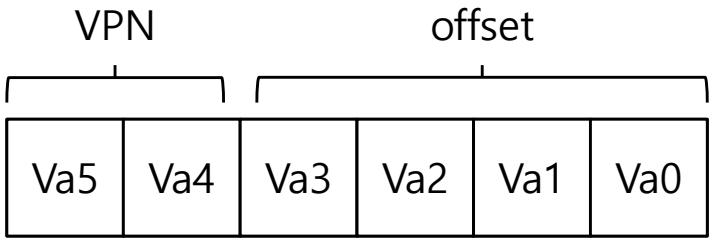
A Simple 64-byte Address Space



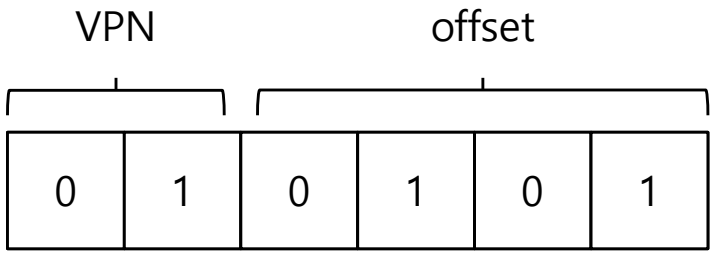
64-Byte Address Space Placed In Physical Memory

Address Translation

- **Two components in the virtual address**
  - VPN: virtual page number
  - Offset: offset within the page

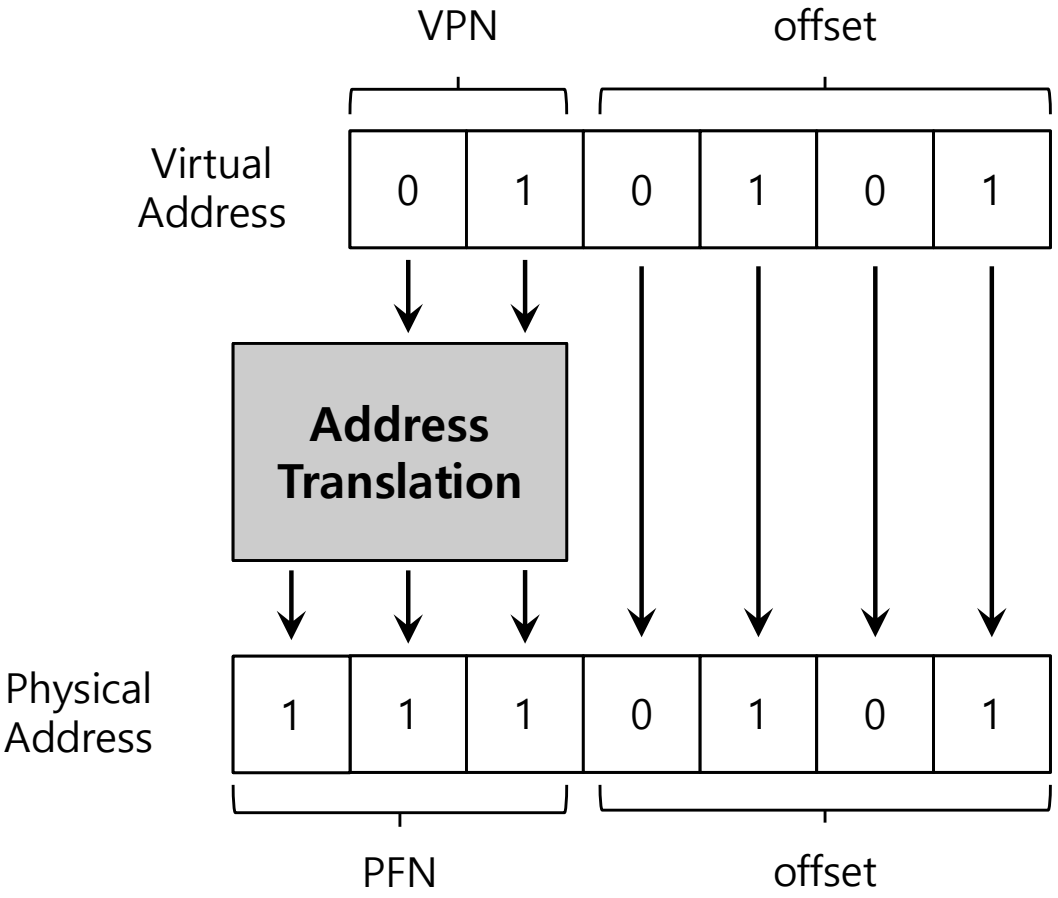


- **Example: virtual address 21 in 64-byte address space**



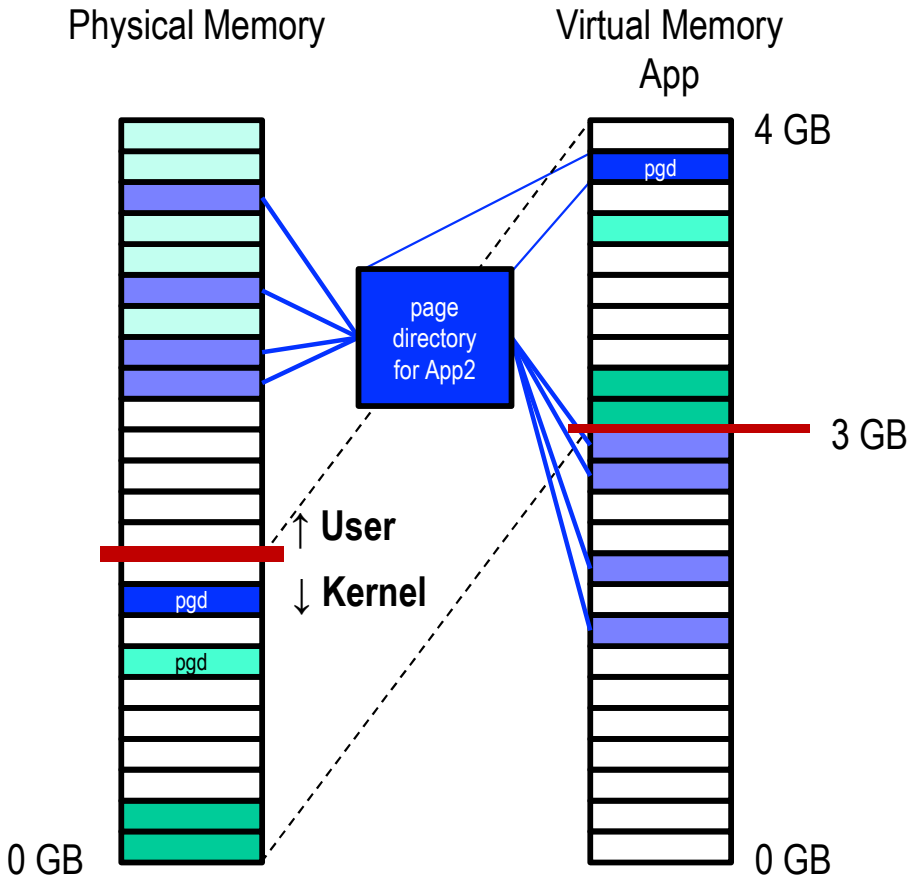
Example: Address Translation

- **The virtual address 21 in 64-byte address space**



Where Are Page Tables Stored?

- **Page tables can get awfully large.**
  - 32-bit address space with 4-KB pages, 20 bits for VPN
    - Page offset for 4 Kbyte page: 20 bit
  - Page table size
    - $4MB = 2^{20}\ entries\ *$ $4\ Bytes\ per\ page\ table\ entry$
- **Page tables for each process are stored in memory.**

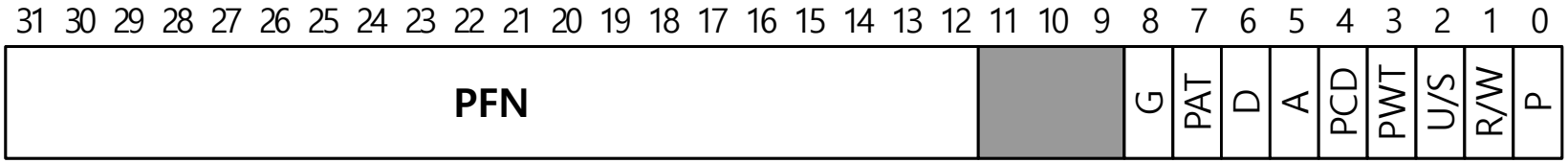


What Is In The Page Table?

- **The page table is a data structure used to map virtual addresses to physical addresses.**
  - The simplest form is a ***linear page table***, implemented as a **contiguous array**.
- **The OS indexes the page table by the Virtual Page Number (VPN) and retrieves the corresponding page-table entry (PTE).**

- **Example:**
  Page Table Entry

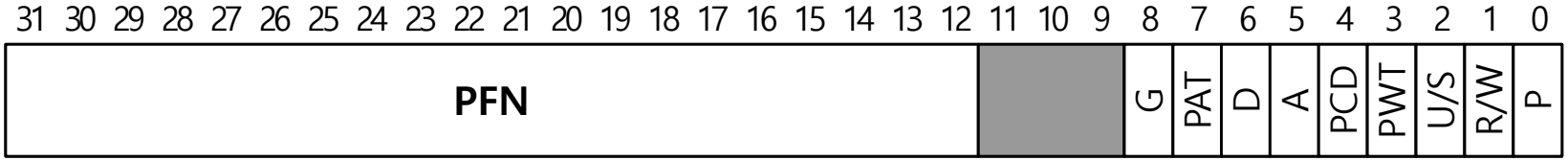
- **P: present**
- **R/W: read/write bit**
- **U/S: supervisor**
- **A: accessed bit**
- **D: dirty bit**
- **PFN: the page frame number**



An x86 Page Table Entry(PTE)

Common Flags of a Page Table Entry (PTE)

- P: present
- R/W: read/write bit
- U/S: supervisor
- A: accessed bit
- D: dirty bit
- PFN: the page frame number



An x86 Page Table Entry(PTE)

- **Present Bit**: Indicates whether the page is currently in physical memory or has been swapped out to disk.
- **Protection Bits**: Specify whether the page can be read, written, or executed.
- **Supervisor Bit**: Indicates whether it is accessable by the kernel or user.
- **Dirty Bit**: Indicates whether the page has been modified since it was loaded into memory.
- **Accessed Bit**(Reference Bit): Indicates whether the page has been accessed recently.

Disadvantages of Paging

- **Paging: Too Slow**
  - To locate the desired PTE, the system must know the starting address of the page table.
  - With paging, every memory reference requires an **additional memory access** to fetch the corresponding page-table entry (PTE).

Accessing Memory With Paging

```
1          // Extract the VPN from the virtual address
2          VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4          // Form the address of the page-table entry (PTE)
5          PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7          // Fetch the PTE
8          PTE = AccessMemory(PTEAddr)
9
10         // Check if process can access the page
11         if (PTE.Valid == False)
12                   RaiseException(SEGMENTATION_FAULT)
13         else if (CanAccess(PTE.ProtectBits) == False)
14                   RaiseException(PROTECTION_FAULT)
15         else
16                   // Access is OK: form physical address and fetch it
17                   offset = VirtualAddress & OFFSET_MASK
18                   PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19                   Register = AccessMemory(PhysAddr)
```

remember:
additional memory access
for address translation

A Memory Trace

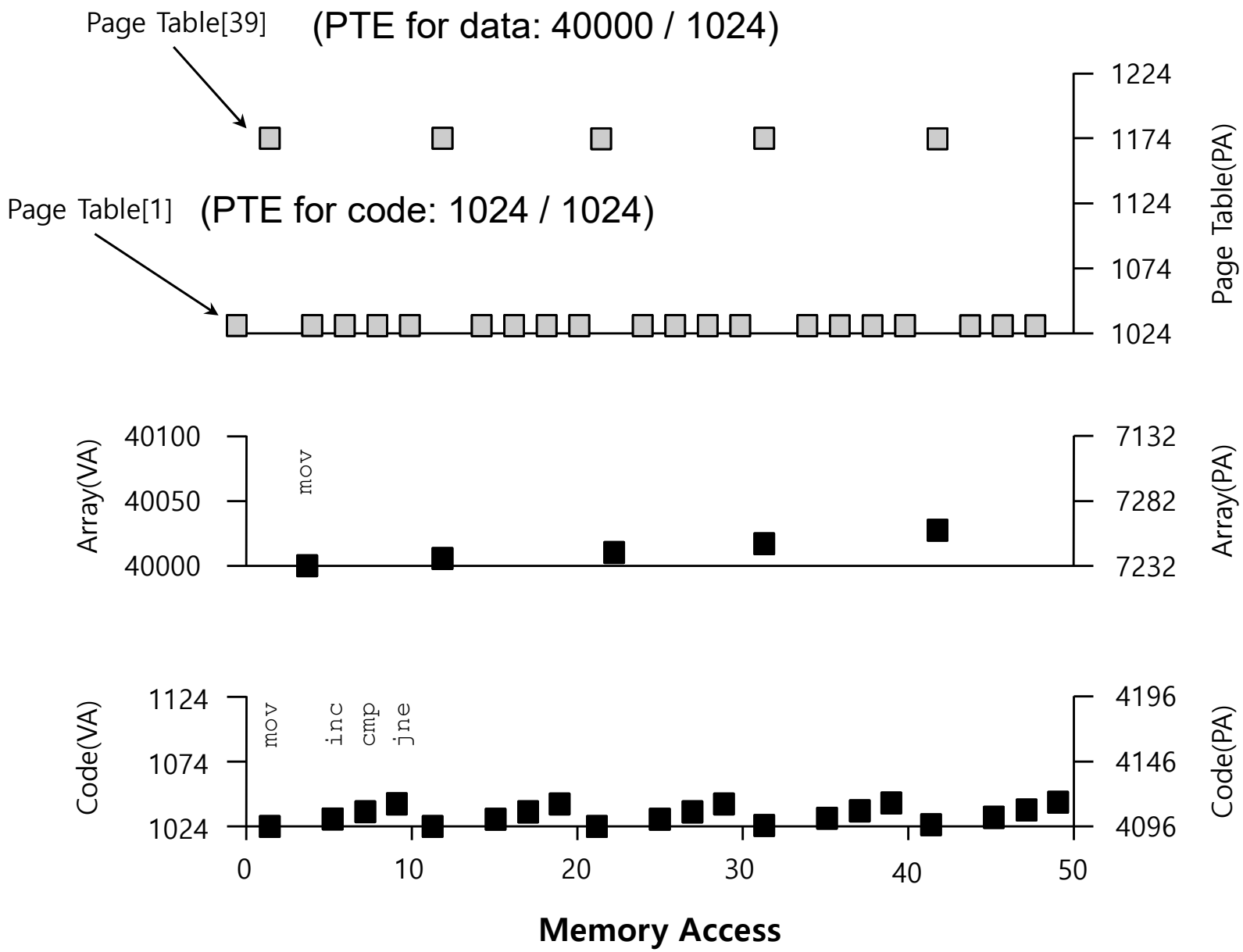
```
int array[1000];
...
for (i = 0; i < 1000; i++)
        array[i] = 0;
```

```
prompt> gcc -o array array.c -Wall -o
prompt>./array
```

Memory access

```
0x1024 movl $0x0,(%edi,%eax,4) //[edi+eax*4]= 0
0x1028 incl %eax
0x102c cmpl $0x03e8,%eax //0000 0011 1110 1000₂ = 1000₁₀
0x1030 jne 0x1024
```

A Virtual(And Physical) Memory Trace



Segmentation and Paging: Summary

- **Segmentation**
  - Divides the address space into **variable-sized logical units** (code, data, heap, stack).
  - Provides **protection** and **sharing** at the segment level.
  - Allows segments to grow independently (e.g., stack, heap).
  - Main drawback: **external fragmentation** and the need for compaction.

- **Paging**
  - Splits virtual memory into **fixed-size pages** and physical memory into **page frames**.
  - Eliminates external fragmentation and simplifies memory allocation.
  - Main drafback: Requires a **page table** to translate VPN → PFN; incurs **extra memory access**.
  - Performance issue alleviated by **TLB**, which caches recent translations.